

Anwendungsorientierte Softwareentwicklung

Laborblatt 6: Bewegte Bilder

Lernziele

- Python-Grundlagen: Tupel
- Erste Schritte mit PyGame
- Weitere Schritte in Richtung objektorientierte Programmierung

Schritt 1: Eigenes Repository klonen

Heute arbeiten wir nur mit Python. Der Mikrocontroller kommt aber später wieder ins Spiel (im wahrsten Sinne des Wortes). Damit euer Code nicht verloren geht, klonet zunächst euer Repository.

Wie immer:

- Öffnet zwei Terminals (links zum Editieren, rechts zum Ausführen von Kommandos).

Linkes Terminal (Editieren)

A) Repository klonen und Verzeichnis wechseln

Klonen des Repositories:

```
git clone https://gitlab.uni-ulm.de/BENUTZERNAME/asp-ulm
```

B) Verzeichnis wechseln:

```
cd asp-ulm/python
```

Rechtes Terminal (Kommandos ausführen)

A) Verzeichnis wechseln

```
cd asp-ulm/python
```

Schritt 2: Python Grundlagen (Listen vs. Tupel)

Ziel: Wir werden im nächsten Schritt das Modul PyGame verwenden, um zu zeichnen. Dabei werden oft sogenannte **Tupel** als Datenstruktur verwendet — zum Beispiel für die Größe des Fensters oder die Position eines grafischen Objekts.

Bevor wir mit PyGame richtig loslegen, machen wir uns mit diesen **Tupeln** und dem Unterschied zu **Listen** vertraut.

Vorbereitung: Startet im rechten Fenster eine Python-Shell mit dem Kommando `python`

1. Zuweisung mit mehreren Variablen

Gebt in der Shell Folgendes ein:

(1) `x, y = 100, 200`

(2) `x`

(3) `y`

Ihr solltet beobachten, dass mit der ersten Anweisung `x` und `y` jeweils der Wert `100` und `200` zugewiesen wird:

```
>>> x, y = 100, 200
```

```
>>> x
```

```
100
```

```
>>> y
```

```
200
```

2. Listen: Veränderbare Datenstruktur

Jetzt erzeugen wir eine Liste `pos` und probieren ein wenig damit herum:

(1) `pos = [x,y]`

(2) `pos[0]`

(3) `pos[1]`

(4) `pos`

(5) `pos.append(42)`

(6) `pos`

Wie ihr aus dem Python Online-Kurs schon kennt, kann man bei einer Liste:

- auf einzelne Elemente zugreifen
- Elemente hinzufügen

In diesem Beispiel beobachtet ihr:

```
>>> pos = [x,y]
```

```
>>> pos[0]
```

```
100
```

```
>>> pos[1]
```

```
200
```

```
>>> pos
```

```
[100, 200]
```

```
>>> pos.append(42)
```

```
>>> pos
```

```
[100, 200, 42]
```

3. Tupel: Unveränderbare Datenstruktur

Jetzt erzeugen wir ein **Tupel** `pos` und beobachten, dass man damit ähnlich arbeiten kann wie mit einer **Liste**.

Allerdings unterstützt ein **Tupel** nur solche Operationen, die **pos** *nicht verändern*. Deshalb verwendet man **Tupel** oft, um feste Werte (z.B. die Größe eines Fensters oder eine feste Position) darzustellen.

Gebt in der Shell Folgendes ein:

```
(1) pos = (x,y)
(2) pos[0]
(3) pos[1]
(4) pos
(5) pos[0] = 3
(6) pos.append(42)
```

Folgendes ist passiert:

- Mit (1) wurde das **Tupel** **pos** mit zwei Elementen erzeugt (man verwendet dafür runde statt eckige Klammern).
- In (2) und (3) wurde auf die Elemente zugegriffen — das funktioniert genauso wie bei einer **Liste**.
- In (5) wurde versucht, das erste Element zu verändern — das ist bei einem **Tupel** *nicht erlaubt*.
- Aus demselben Grund wird auch kein **.append()** unterstützt, um ein Tupel zu erweitern.

Beispielausgabe:

```
>>> pos = (x,y)
>>> pos[0]
100
>>> pos[1]
200
>>> pos
(100, 200)
>>> pos[0] = 3
Traceback (most recent call last):
  File "<python-input-27>", line 1, in <module>
    pos[0] = 3
    ~~~^^^
TypeError: 'tuple' object does not support item assignment
>>> pos.append(42)
Traceback (most recent call last):
  File "<python-input-28>", line 1, in <module>
    pos.append(42)
    ^^^^^^^^^^^
AttributeError: 'tuple' object has no attribute 'append'
```

Fazit:

- **Tupel** eignen sich für *feste Werte*, die sich im Programm nicht ändern sollen.
- In *PyGame* werdet ihr z.B. häufig Tupel verwenden, um die Fenstergröße oder eine feste Position zu übergeben.

Schritt 3: Erste Schritte mit PyGame

Ziel: In diesem Schritt werden wir die **SDL**-Bibliothek (*Simple DirectMedia Layer*) verwenden.

SDL ist eine in C geschriebene Bibliothek, die es ermöglicht, direkt auf die **Grafikkarte**, den **Sound** und andere **Hardware-Komponenten** zuzugreifen. Damit eignet sie sich hervorragend, um Spiele oder andere grafische Anwendungen zu programmieren.

Allerdings werden wir **SDL nicht direkt**, sondern über die **Python-Bibliothek PyGame** benutzen.

PyGame ist ein sogenannter **Wrapper** für SDL: Es stellt uns eine komfortable Python-Schnittstelle zur Verfügung, mit der wir SDL sehr einfach und effizient nutzen können.

In diesem Schritt lernen wir zunächst:

- wie man mit PyGame ein **Fenster mit einer bestimmten Größe** erzeugt und
- wie man darin ein **grafisches Objekt** (hier: einen gelb gefüllten Kreis als "Ball") zeichnet.

Dabei geht es vor allem darum, zu verstehen, wie ein typisches PyGame-Programm **strukturell aufgebaut** ist:

- Ein solches Programm besteht meist aus einer **Initialisierung** und einer **Hauptschleife (Loop)**, in der jeweils eine **Szene** gezeichnet wird.
- Das ist die **Grundstruktur**, die wir später Schritt für Schritt erweitern werden — zum Beispiel, um den Ball **zu bewegen**.

Vorbereitung:

Verwendet im linken Terminal den Editor eurer Wahl, um eine Datei `breakout.py` mit folgendem Inhalt zu erzeugen. (Die leeren Zeilen sind Absicht – in den Aufgaben werden wir diese nach und nach ausfüllen.)

```
1 import pygame
2
3
4 # setup
5 pygame.init()
6
7 width, height = 800, 600
8 screen = pygame.display.set_mode((width, height))
9
10 ball_c = (255, 255, 0)
11 ball_x, ball_y = 400, 300
12
13 # draw stuff
14
15 pygame.draw.circle(screen, ball_c, (ball_x, ball_y), 5)
16
17
18
19 # done
20 pygame.quit()
```

Aufgaben

1. Führt das Programm im rechten Terminal mit dem Befehl `python breakout.py` aus. Es sollte kurz ein neues Fenster erscheinen, das jedoch blitzartig wieder verschwindet.

2. Bisher gab es noch keine Erklärung, was die einzelnen Programmzeilen bedeuten. Das ist im Moment aber auch wenig interessant, solange man im Fenster nichts sieht.

Kümmern wir uns also zunächst darum, dass das Fenster nicht sofort verschwindet:

1. Fügt in **Zeile 2** folgende Zeile ein: `import time`

2. Fügt in **Zeile 17** folgende Zeile ein: `time.sleep(5)`

Wenn ihr jetzt das Programm erneut mit `python breakout.py` startet, bleibt das Fenster für 5 Sekunden sichtbar.

Allerdings ist das Fenster noch schwarz und enthält keinen sichtbaren Inhalt.

Damit das sichtbare Ergebnis ein wenig interessanter wird, fügt in **Zeile 16** folgende Zeile ein:

`pygame.display.flip()`

Führt das Programm nochmals aus. Jetzt solltet ihr während der 5 Sekunden im neuen Fenster in der Mitte einen gelben Kreis sehen.

Was das Programm macht:

- Mit `import pygame` wird das Modul `pygame` importiert.
- Mit `pygame.init()` werden die nötigen Vorbereitungen getroffen, damit man überhaupt mit der Grafikkarte, der Tastatur oder der Soundkarte arbeiten kann.
- Mit `pygame.display.set_mode` wird ein Fenster erzeugt, in das man anschließend zeichnen kann. Der Funktion wird die Breite und Höhe des Fensters übergeben – als sogenanntes Tupel, also ein Zahlenpaar wie `(800, 600)`.

Die Funktion gibt eine **Variable** zurück, mit der man später in dieses Fenster zeichnen kann. Genauer gesagt handelt es sich dabei um ein sogenanntes **Objekt** vom Typ `Surface` – was letztlich auch nichts anderes als eine spezielle Variable ist, die `pygame` intern verwendet, um das Fenster zu verwalten. Für uns reicht es zu wissen: Diese Variable wird später als erstes Argument an Zeichenfunktionen wie `pygame.draw.circle` übergeben.

- Mit `pygame.draw.circle` kann man einen Kreis zeichnen. Die Funktion erwartet vier Argumente:
 - die Variable, die das Fenster repräsentiert (also: *wohin* gezeichnet werden soll),
 - die Farbe des Kreises – hier ein RGB-Tupel wie `(255, 255, 0)` für Gelb,
 - den Mittelpunkt des Kreises – als Zahlenpaar `(x, y)` für die Position auf dem Bildschirm,
 - den Radius des Kreises – also wie groß er sein soll.
- `pygame` (bzw. das darunterliegende **SDL**) verwendet eine Technik namens **Double Buffering**. Man kann sich das vorstellen wie zwei Zeichenflächen: Eine davon sieht man, die andere ist „hinter den Kulissen“. Man zeichnet immer auf die verborgene Fläche, und wenn alles fertig ist, schaltet `pygame.display.flip()` auf die neue Fläche um. So verhindert man, dass man dem Programm beim Zeichnen zusehen kann – bei Animationen würde das sonst zu einem hässlichen Flackern oder Ruckeln führen.
- Am Ende ruft das Programm `pygame.quit()` auf. Das ist nicht unbedingt nötig, weil beim Beenden sowieso alles aufgeräumt wird. Aber es ist guter Stil, die von `pygame` belegten Ressourcen wieder freizugeben.

Zum Abschluss dieses Schrittes:

Fügt die Datei zum Repository hinzu, erzeugt einen Commit und pusht ihn mit folgenden Befehlen:

```
git add breakout.py
git commit -a -m "first steps with pygame"
git push
```

Schritt 4: Struktur eines PyGame-Programms und bewegte Bilder

Ziel: Unser PyGame-Programm soll nicht einfach nur eine bestimmte Anzahl von Sekunden laufen, sondern so lange, bis es vom Benutzer beendet wird — also entweder indem das Fenster (durch Klicken auf das **X** oben rechts) geschlossen wird oder indem eine bestimmte Taste, zum Beispiel **Q**, gedrückt wird.

Vorbereitung: Unten seht ihr einen leicht angepassten Code für `breakout.py` mit **didaktischen Leerzeilen**, die wir in den Aufgaben schrittweise füllen werden. Die wesentliche Änderung ist die **while-Schleife** in Zeile 16. Diese läuft, solange die Variable `running` den Wert **True** hat. In dieser Version ist `running` zunächst immer **True**. Die Anweisungen in den Zeilen 17 bis 28 werden also in einer **Endlos-Schleife** immer wieder ausgeführt. Eine weitere Änderung ist, dass in Zeile 28 nach dem Sichtbarmachen der Zeichenfläche (`pygame.display.flip()`) eine Pause von **1/30 Sekunde** eingebaut ist. Damit wird das Bild **mit ca. 30 Bildern pro Sekunde (FPS – Frames per Second)** aktualisiert. Hier der vorbereitete Code:

```
1 import pygame
2 import time
3
4 # setup
5 pygame.init()
6
7 width, height = 800, 600
8 screen = pygame.display.set_mode((width, height))
9
10 ball_c = (255, 255, 0)
11 ball_x, ball_y = 300, 300
12
13
14 # draw stuff
15 running = True
16 while running:
17
18
19
20
21
22
23
24     pygame.draw.circle(screen, ball_c, (ball_x, ball_y), 5)
25
26
27     pygame.display.flip()
28     time.sleep(1/30)
29
30 # done
31 pygame.quit()
```

Aufgaben

1. Passt `breakout.py` wie oben gezeigt an und testet es im rechten Terminal. Das Programm läuft jetzt in einer **Endlos-Schleife** und kann im Moment nur beendet werden, indem ihr es mit **Control-C** im Terminal **abbreht**.

2. Damit man das Programm auch beenden kann, indem das Fenster geschlossen wird, fügt folgende Zeilen ein. Achtet darauf, dass die Zeilen so eingerückt sind, dass sie zum while-Block der Schleife in Zeile 16 gehören:

```
17     for event in pygame.event.get():
18         if event.type == pygame.QUIT:
19             running = False
```

Die Funktion `pygame.event.get()` liefert eine **Liste von sogenannten Events**. Ein Event stellt ein Ereignis dar — zum Beispiel das Schließen eines Fensters, das Drücken einer Taste usw. Testet das Programm, bevor ihr anschließend einen ersten **Crash-Kurs in "Was ist ein Objekt?"** bekommt.

An dieser Stelle kann man es kaum noch verheimlichen: Events werden als **Objekte** repräsentiert. Der Begriff Objekt wurde bisher meist vermieden und stattdessen von Variablen gesprochen. Das ist auch nicht falsch — ein Objekt ist im Grunde eine **spezielle Art von Variable**.

Was macht Objekte speziell? Ein Objekt kann **aus mehreren Komponenten** bestehen. Man kann sich das vorstellen wie ein **Bündel von Werten**, die zusammengehören. Jede dieser Komponenten kann wiederum selbst eine Variable oder ein weiteres Objekt sein. Der gängige Begriff für diese Komponenten lautet **Attribute**.

Ein solches **Attribut** eines Event-Objekts ist z.B. `type`. Wenn ein Event das **Schließen eines Fensters** darstellt, dann hat dieses Attribut den Wert `pygame.QUIT`. Der obige Code bewirkt in diesem Fall, dass `running` auf `False` gesetzt wird — dadurch wird die **äußere while-Schleife** verlassen, und das Programm wird beendet.

3. Wird auf der Tastatur eine Taste gedrückt, dann erzeugt dies ein **Event-Objekt**, dessen Attribut `type` den Wert `pygame.KEYDOWN` hat. In einem weiteren Attribut `key` wird hinterlegt, welche Taste gedrückt wurde. Beispielsweise repräsentiert der Wert `pygame.K_q` die Taste **Q**. Ändert das Programm wie folgt:

```
17     for event in pygame.event.get():
18         if event.type == pygame.QUIT:
19             running = False
20         elif event.type == pygame.KEYDOWN:
21             if event.key == pygame.K_q:
22                 running = False
```

Jetzt sollte es auch möglich sein, das Programm zu beenden, indem man die Taste **Q** drückt.

4. Damit etwas **Bewegung** ins Spiel kommt, ändern wir in jedem Frame die **Position des Balls**. Zunächst legen wir für den Ball eine **Geschwindigkeit** in x- und y-Richtung fest:

```
12 ball_vx, ball_vy = -1, 2
```

Die Einheit für die Geschwindigkeit ist **"Pixel pro Frame"**, denn mit den Zeilen

```
25     ball_x += ball_vx
26     ball_y += ball_vy
```

wird die **Position** des Balls nach ca. **1/30 Sekunde** jeweils aktualisiert. Testet das Programm nach dieser Änderung — der verbleibende **Schönheitsfehler** wird im Anschluß korrigiert.

5. Der Ball hinterlässt bei seiner Bewegung noch Spuren. Die Lösung dazu ist simpel, wenn man einen neuen Frame zeichnet, dann beginnt man damit die Leinwand zunächst zu leeren. Konkret heisst das bei uns, dass wir sie komplett schwarz einfärben. Der RGB-Wert für schwarz ist (0,0,0) und mit folgender Zeile wird die Zeichenfläche schwer eingefärbt:

```
23     pygame.Surface.fill(screen, (0, 0, 0))
```

Der Ball sollte sich jetzt bewegen ohne Spuren zu hinterlassen.

6. Der Ball fliegt im Moment noch **ins Nirgendwo**. Mit der nächsten Änderung soll er an den **Rändern** des Fensters reflektiert werden. Ab jetzt werden im Programm keine Lücken mehr gefüllt, sondern **neue Zeilen hinzugefügt**.

Fügt direkt **nach Zeile 26** folgende Zeilen hinzu:

```
27     if ball_x < 0 or ball_x > width:
28         ball_vx *= -1
29     if ball_y < 0 or ball_y > height:
30         ball_vy *= -1
```

Die Reflexion ist noch nicht perfekt, aber für's Erste gut genug:

- Hat der Ball das Spielfeld **links oder rechts** verlassen, wird die Geschwindigkeit in **x-Richtung** umgekehrt.
- Hat der Ball das Spielfeld **oben oder unten** verlassen, wird die Geschwindigkeit in **y-Richtung** umgekehrt.

Für eine **perfekte Reflexion** müsste man nicht nur die Geschwindigkeit anpassen, sondern auch die **Position** korrekt setzen.

Aber das ist einfache Mathematik — das können wir später noch ergänzen.

7. Der Ball fliegt und prallt — prima! Jetzt brauchen wir noch ein **Paddle**, also ein **Rechteck**. Das ist eine schöne Gelegenheit, das gerade Gelernte zu wiederholen und zu variieren. Ein **Paddle** ist in PyGame einfach ein **Rechteck**, das durch:

- die Koordinaten der **linken oberen** Ecke sowie
 - eine **Breite** und **Höhe**
- beschrieben wird.

Natürlich kann man auch einem Rechteck eine **Farbe** geben. Dazu legen wir zunächst ein paar **Variablen** an. Fügt folgende Zeilen **nach Zeile 12** hinzu (analog zu den Variablen für den Ball):

```
13 paddle_c = (0, 0, 255)
14 paddle_x, paddle_y = 300, 570
15 paddle_w, paddle_h = 100, 30
```

In jedem Frame soll das Paddle gezeichnet werden. Fügt dazu **nach Zeile 27** (also **nachdem der Ball gezeichnet wurde**) folgende Zeilen ein:

```
28     pygame.draw.rect(screen, paddle_c,
29                       (paddle_x, paddle_y, paddle_w, paddle_h))
```

8. Das **Paddle** sollte man natürlich **bewegen können**. Später werden wir dazu die **serielle Schnittstelle** auslesen und können dann durch Drehen am Potentiometer die Position verändern. Für heute sollen zum Steuern die Pfeiltasten **links** und **rechts** genügen. Wir fragen im Code ja bereits ab, ob die Taste **Q** gedrückt wurde. Erweitert diese Abfrage wie folgt:

```
23         elif event.type == pygame.KEYDOWN:
24             if event.key == pygame.K_q:
25                 running = False
26             elif event.key == pygame.K_LEFT:
27                 paddle_x -= 10
28             elif event.key == pygame.K_RIGHT:
29                 paddle_x += 10
```

Testet das Programm nach dieser Änderung. Ihr solltet dabei zwei Dinge beobachten, die noch stören:

- Das Paddle kann aus dem Fenster heraus bewegt werden.
- Wenn man die Pfeiltasten gedrückt hält, ändert sich die Position nicht kontinuierlich — man muss die Taste jedes Mal loslassen und neu drücken.

9. Es wäre angenehmer, wenn man eine **Pfeiltaste gedrückt hält**, dass das Paddle dann so lange in die entsprechende Richtung bewegt wird, bis man die Taste wieder loslässt. Entfernt

dazu zunächst wieder die in der vorigen Teilaufgabe eingefügten Zeilen (bei mir waren das die Zeilen **26 bis 29**).

Tastatur-Events werden nur dann erzeugt, wenn eine Taste **gedrückt** oder **losgelassen** wird — **nicht**, wenn eine Taste gedrückt **gehalten** wird. Würde es dafür Events geben, würde das viel zu viele Events erzeugen.

Man kann aber abfragen, **welche Tasten aktuell gedrückt gehalten werden**, und dann prüfen, ob die **Pfeiltasten** dabei sind.

Fügt dazu folgende Zeilen **nach der for-Schleife** ein. Dabei ist es wichtig, dass diese Zeilen **zur while-Schleife** gehören, aber **nicht zur for-Schleife** für die Events! Die Zeilen sollen ja auch dann ausgeführt werden, wenn kein Event vorliegt.

→ Achtet also genau auf die **Einrücktiefe** (bei mir 4 Zeichen).

```
26     keys = pygame.key.get_pressed()
27     if keys[pygame.K_LEFT]:
28         paddle_x -= 10
29     if keys[pygame.K_RIGHT]:
30         paddle_x += 10
```

Mit der Funktion `get_pressed()` in Zeile 26 erhält man in `keys` ein sogenanntes **Dictionary**. Im Python-Brückenkurs wird diese Datenstruktur noch genauer behandelt. Hier reicht es zu wissen, dass man sie ähnlich benutzen kann wie eine **Liste** — nur dass der **Index** keine Zahl sein muss.

Zum Beispiel wird hier mit `keys[pygame.K_LEFT]` und `keys[pygame.K_RIGHT]` auf Einträge von `keys` zugegriffen. Diese Werte sind entweder **True** oder **False** — je nachdem, ob die entsprechende Taste gerade **gedrückt ist oder nicht**.

10. Jetzt lösen wir noch das Problem, dass das **Paddle** derzeit noch **zu weit nach links** oder **nach rechts** bewegt werden kann.

Das geht ganz einfach mit folgendem **Fix**, den ihr **nachdem die Position des Paddles eventuell verändert wurde** einfügen solltet:

```
31     if paddle_x < 0:
32         paddle_x = 0
33     if paddle_x + paddle_w > width:
34         paddle_x = width - paddle_w
```

Zum Abschluss dieses Schrittes:

Fügt die Datei zum Repository hinzu, erzeugt einen Commit und pusht ihn mit folgenden Befehlen:

```
git commit -a -m "ball and paddle hack"
git push
```

Was als Nächstes kommt

Man könnte jetzt einfach weiter am Spiel arbeiten:

- Zum Beispiel dafür sorgen, dass das **Paddle** den **Ball tatsächlich abprallen lässt**.
- Oder, dass man den **Ball verliert**, wenn er unten aus dem Spielfeld herausfliegt.

Aber leider spiele ich an dieser Stelle ein wenig den **Spielerverderber** — und zeige euch in den nächsten Schritten stattdessen, **wie man den Code besser organisiert**.

Dazu führen wir eine **Klasse Paddle** und eine Klasse **Ball** ein.

Mit dieser Struktur lassen sich weitere Features später **viel einfacher und übersichtlicher umsetzen**.

Schritt 5: Klasse Paddle

Unser **Paddle** besteht aus **Daten** und **Operationen**, die diese Daten manipulieren:

- **Daten** sind die Variablen: `paddle_x`, `paddle_y`, `paddle_w`, `paddle_h`, `paddle_c`.
- **Operationen** sind der Code, der das Paddle **nach links oder rechts bewegt**, sowie der Code, um das Paddle **zu zeichnen**.

In der aktuellen Fassung ist all das kreuz und quer im Code verstreut. Vom letzten Laborblatt ist euch vielleicht noch in Erinnerung geblieben, dass man **globale Variablen** in einer **Klasse** kapseln kann. Und in die Klasse kann man auch **Funktionen** packen, die diese Variablen manipulieren.

Auf das Paddle angewendet könnte das so aussehen:

```

14 class Paddle:
15     c = (0, 0, 255)
16     w, h = 100, 30
17     x, y = (width - w) // 2, 570
18
19     def left():
20         Paddle.x -= 10
21         if Paddle.x < 0:
22             Paddle.x = 0
23
24     def right():
25         Paddle.x += 10
26         if Paddle.x + Paddle.w > width:
27             Paddle.x = width - Paddle.w
28
29     def draw(surface):
30         pygame.draw.rect(surface, Paddle.c,
31                          (Paddle.x, Paddle.y, Paddle.w, Paddle.h))

```

Ersetzt die Zeilen in denen zuvor `paddle_x`, `paddle_y`, `paddle_w`, `paddle_h` und `paddle_c` initialisiert wurden mit diesem Code.

Nochmals zur **Klarstellung**, was dieser Code bewirkt:

- Damit werden globale Variablen `Paddle.c`, `Paddle.x`, `Paddle.y`, `Paddle.w`, `Paddle.h` initialisiert. Man könnte auch sagen, dass man damit Klassenattribute `c`, `x`, `y`, `w`, `h` von `Paddle` definiert.
- Es werden die **Funktionen** `Paddle.left` und `Paddle.right` definiert, um diese Variablen so zu verändern, dass sich das Paddle nach **links** oder **rechts** bewegt.
- Es wird die **Funktion** `Paddle.draw` definiert, um das Paddle an seiner aktuellen Position auf eine Zeichenfläche zu zeichnen.
- Die in der Klasse definierten Funktionen könnte man auch als **Klassen-Methoden** `left`, `right` und `draw` von `Paddle` bezeichnen.

Aufgabe: Make use of it!

Ändert den Rest des Programms so, dass das **Paddle** über die **Klassenmethoden** gesteuert wird:

- soll sich das Paddle **nach links bewegen** ruft `Paddle.left()` auf.
- soll sich das Paddle **nach rechts bewegen** ruft `Paddle.right()` auf.
- Und ruft `Paddle.draw(screen)`, um das Paddle auf die Zeichenfläche `screen` zu zeichnen.

Das Programm wird dadurch nicht besser (der Ball fliegt immer noch durch das Paddle), aber lesbarer. Wenn alles klappt:

```

git commit -a -m "paddle class"
git push

```

Schritt 5: Grundlagen zu Klassen mit mehreren Instanzen

Für den **Ball** könnten wir — wie für das Paddle — ebenfalls eine **Klasse** definieren. Dabei nutzen wir jetzt die Gelegenheit, sogenannte **instanzierbare Klassen** einzuführen. Das klingt zunächst **fancy**, ist aber ganz einfach:

Es bedeutet nur, dass man von einer solchen Klasse **mehrere Objekte** erzeugen kann.

Oder anders gesagt:

→ Es kann **mehrere Bälle** geben — aber nur ein **Paddle**.

Die **Grundidee** erkunden wir zunächst in der **interaktiven Python-Shell**. Startet diese im **rechten Terminal** mit dem Kommando `python` und gebt zunächst folgendes ein, um eine „leere“ Klasse Ball zu definieren:

```
class Ball:  
    pass
```

Danach:

```
(1) ball = Ball()  
(2) print(type(ball))  
(3) ball.x = 42  
(4) print(ball.x)
```

Ihr solltet dabei beobachten:

- Mit (1) wurde eine neue Variable (aka **Objekt**) `ball` definiert. Wie sich in (2) zeigt, hat diese den Typ `Ball`.
- In (3) wurde dem Objekt `ball` ein **Attribut** `x` hinzugefügt und mit `42` initialisiert.
- In (4) wird dieses Attribut `x` ausgegeben.

In eine Klasse kann man auch Funktionen packen, die dem Objekt:

- Attribute anhängen
- diese manipulieren oder
- diese auslesen.

Gebt zunächst folgendes ein:

```
class Ball:  
    def init(ich, x):  
        ich.x = x  
    def print(ich):  
        print(f"mein x ist {ich.x}")
```

Danach

```
(1) ball = Ball()  
(2) Ball.init(ball, 123)  
(3) Ball.print(ball)
```

Ihr solltet beobachten:

- Jetzt wird `ball.x` mit `123` initialisiert.
- Das Attribut `x` wird anschließend ausgegeben.
- Der Vorteil: Das Attribut `x` wird hier nur **innerhalb der Funktionen** verwendet und **manipuliert**.

Jetzt erzeugen wir einen **zweiten Ball** und rufen die Funktionen `init` und `print` diesmal anders auf:

- (1) `ball2 = Ball()`
- (2) `ball2.init(456)`
- (3) `ball2.print()`

Ihr solltet beobachten:

- `ball2.init(456)` hat den gleichen Effekt wie `Ball.init(ball2, 456)`.
- `ball2.print()` hat den gleichen Effekt wie `Ball.print(ball2)`.

Tatsächlich wird das in Python intern genau so **umgeformt**:

Beim Aufruf `ball2.print()` wird im Hintergrund automatisch `Ball.print(ball2)` daraus gemacht.

Jetzt versuchen wir, einen dritten Ball anzulegen — vergessen dabei aber, ihn vorher zu initialisieren, bevor wir ihn ausgeben. Gebt ein:

- (1) `ball3 = Ball()`
- (2) `ball2.print()`

Ihr werdet sehen: Das gibt einen **Fehler**.

Denn `ball3.x` wurde noch nicht gesetzt — wir haben `init()` diesmal nicht aufgerufen.

Wäre es nicht eine gute Idee, wenn eine **Initialisierung automatisch** erfolgt, sobald man einen neuen Ball anlegt?

→ Diese Idee hatte tatsächlich schon jemand — und sie ist in **Python** direkt eingebaut!

Gebt folgendes ein (bzw. verwendet **Pfeil-hoch**, um den alten Code für die Klasse `Ball` zurückzuholen und ändert ihn ab):

```
class Ball:
    def __init__(ich, x):
        ich.x = x
    def print(ich):
        print(f"mein x ist {ich.x}")
```

Gebt dann ein:

- (1) `ball3 = Ball(321)`
- (2) `ball2.print()`

Ihr seht jetzt:

Die Initialisierung mit `__init__` passiert automatisch, sobald ihr ein neues Objekt `Ball(321)` erzeugt. Ihr müsst `init` also nicht mehr manuell aufrufen — das übernimmt Python **automatisch**.

Üblicherweise verwendet man nicht `ich`, sondern `self` als **ersten Parameter** der Funktionen (auch genannt **Objekt-Methoden**), der das Objekt angibt, dessen Attribute **initialisiert**, **manipuliert** oder **ausgelesen** werden.

Das ist allerdings nur **Konvention** — man könnte theoretisch jeden anderen Namen wählen.

Nicht verhandelbar ist aber, dass die Funktion zur automatischen Initialisierung (auch **Konstruktion** genannt) den Namen `__init__` haben muss.

Schritt 6: Die richtige Klasse Ball

Das Konzept einer **Klasse**, von der man **mehrere Instanzen** konstruieren kann, lässt sich jetzt für unser Spiel wie folgt umsetzen:

```

10 class Ball:
11     def __init__(self, x, y, vx, vy):
12         self.x = x
13         self.y = y
14         self.vx = vx
15         self.vy = vy
16         self.c = (255, 255, 0)
17
18     def move(self):
19         self.x += self.vx
20         self.y += self.vy
21         if self.x < 0 or self.x > width:
22             self.vx *= -1
23         if self.y < 0 or self.y > height:
24             self.vy *= -1
25
26     def draw(self, surface):
27         pygame.draw.circle(surface, self.c, (self.x, self.y), 5)

```

Ersetzt damit die Zeilen, in denen zuvor die Variablen `ball_c`, `ball_x`, `ball_y`, `ball_vx` und `ball_vy` initialisiert wurden.

Kurze Erklärung der Klasse:

- Mit dem Konstruktor `__init__` kann man ein neues **Ball-Objekt** erzeugen und dabei dessen **Anfangsposition** und **Anfangsgeschwindigkeit** festlegen. Intern werden die entsprechenden **Attribute** ans Objekt angehängt — zusätzlich ein **Attribut für die Farbe** mit einem **Default-Wert**.
- Mit der Methode `move` kann ein Ball-Objekt **bewegt** werden. An den Rändern wird es wie zuvor **reflektiert**.
- Mit der Methode `draw` kann ein Ball-Objekt auf eine Zeichenfläche gezeichnet werden.

Aufgabe: Make use of it!

- Jedes Feature, das man programmiert, sollte man auch **verwenden**. Wenn wir also eine **Klasse** haben, von der man **mehr als eine Instanz** erzeugen kann, dann sollte man mindestens **zwei Instanzen** erzeugen! Fügt dazu folgende Zeile **vor der while-Schleife** ein:

```
49 balls = [Ball(400, 300, -1, 2), Ball(200, 100, 4, 2)]
```

Damit enthält die **Liste** `balls` zwei **Ball-Objekte** mit unterschiedlichen **Anfangspositionen** und **Anfangsgeschwindigkeiten**.

- Wenn die Bälle **gezeichnet** werden, kann man sie **anschließend auch gleich bewegen**. Um das auf **alle Ball-Objekte** anzuwenden, könnt ihr über die Elemente von `balls` iterieren:

```

66     for ball in balls:
67         ball.draw(screen)
68         ball.move()

```

Ersetzt damit die Zeile, mit der bisher der **Ball gezeichnet** wurde. Entfernt außerdem den **alten Code**, mit dem bisher der Ball bewegt wurde (also Code, der noch `ball_x`, `ball_y`, `ball_vx`, `ball_vy` verwendet hat).

- Wenn alles klappt: `git commit -a -m "ball class"` und dann `git push`